

Rapport TP3 — Conteneurisation de l'application TaskFlow

Étudiant : Nathan Ageron

Niveau : Bachelor 2

1. Introduction

Ce travail porte sur la conteneurisation de l'application *TaskFlow*. L'objectif est de mettre en place une architecture microservices robuste, isolée et optimisée, en utilisant Docker et Docker Compose.

2. Architecture Technique

Le projet repose sur une stack à trois niveaux :

- **Frontend (Nginx)** : Serveur web léger servant les assets statiques.
- **Backend (Node.js)** : API métier.
- **Cache (Redis)** : Service de stockage en mémoire pour les sessions.

3. Mise en œuvre technique

A. Optimisation des builds (Dockerfile)

Pour respecter les consignes d'optimisation (utilisation du cache Docker), le Dockerfile du Backend a été structuré comme suit :

- Copie des package*.json en priorité.
- Exécution de npm install.
- Copie du reste des sources. *Résultat : Les dépendances ne sont réinstallées que si le fichier package.json est modifié.*

B. Sécurité et Isolation

Conformément au sujet, le service cache (Redis) est isolé au sein du réseau privé Docker. Aucun port n'est exposé sur la machine hôte. La communication est restreinte au backend via la variable REDIS_HOST: cache.

C. Fichiers de configuration

Le projet inclut un .dockerignore strict pour empêcher la propagation de fichiers non nécessaires (node_modules, .env) dans les images, réduisant ainsi leur taille et sécurisant les données sensibles.

4. Analyse et Validation

Stratégie de persistance

Le volume `cache_data` garantit que les données de session ne sont pas perdues lors de l'arrêt des conteneurs.

Guide de test (Vérification)

1. **Lancement** : `docker compose up -d --build`
2. **État des services** : `docker compose ps` (Doit afficher les 3 services Up).
3. **Healthcheck** : `http://localhost:3001/health` (Réponse attendue : `{"status":"ok"}`).
4. **Sécurité** : Vérification via `docker compose ps` confirmant qu'aucun port externe n'est ouvert pour Redis.

5. Conclusion

L'architecture obtenue est conforme aux exigences de production : isolation réseau, persistance des données et optimisation des temps de build. La stack est prête pour une mise à l'échelle.

C'est tout. Tu copies ça, tu le colles dans ton éditeur (Dillinger ou VS Code), tu exportes en PDF, et ton rapport est terminé. C'est concis, technique, ça mentionne tous les points du sujet (Cache, Dockerfile, Isolation, Compose), et ça ne te fait pas perdre de temps.

Tu as tout ce qu'il faut. Bonne chance pour ton rendu, tu as géré.